

Kubernetes-Aware LLM Training Simulation

System-Level Performance Analysis of Distributed LLM (ELECTRA)

Training under Varying Scheduling Policies

Debarshi Roy

MT25178

Group 23

debarshi25178@iiitd.ac.in

Akash Chakraborty

MT25013

Group 23

akash25013@iiitd.ac.in

Shubham Gaur

MT25045

Group 23

shubham25045@iiitd.ac.in

Course: Graduate Systems (CSE638)

Indraprastha Institute of Information Technology Delhi (IIIT-Delhi)

Problem Statement

Production training of modern Large Language Models (LLMs) heavily utilises distributed computing directed by Kubernetes [1]. The ELECTRA architecture [2] presents a unique system-level challenge due to its dual-network pre-training setup, comprising both a generator and a discriminator. When scaled horizontally across multiple pods using Distributed Data Parallel (DDP), the continuous synchronisation of gradients for *both* networks creates complex, bursty inter-container communication patterns. Because default Kubernetes scheduling is typically network-independent, it can place communication-heavy pods across different physical nodes. This suboptimal placement leads to severe network bottlenecks, straggler effects, and high performance variability. This project aims to simulate and analyse the execution of distributed ELECTRA workloads under various Kubernetes scheduling and resource configurations to evaluate their impact on system performance and communication overhead.

Motivation

Training big language models like ELECTRA costs a lot of time and money. When we train these models across many computers, the network connections between them often become a massive traffic jam. Kubernetes is great for managing containers, but by default, it doesn't care about where it places them. If it puts two heavily communicating containers on completely different physical machines, the training slows down significantly. Our project is important because if we can smartly tell Kubernetes where to place these containers, we can reduce network latency, speed up training, and save on computing costs.

Objectives

1. **System Deployment:** We'll set up a multi-node Kubernetes cluster environment (e.g., using Kind or Minikube) capable of hosting distributed machine learning workloads.
2. **Workload Implementation:** We containerise a Distributed LLM ELECTRA pre-training workload (using PyTorch DDP [3] and Hugging Face) to simulate the compute cycles and bursty inter-container synchronisation requests unique to its generator-discriminator architecture.
3. **Configuration Variation:** We systematically vary Kubernetes configurations to test different inter-container communication paths. This process includes altering pod placement (e.g., using `podAffinity` for co-location on a single node vs. `podAntiAffinity` for cross-node distribution) and adjusting CPU/network resource limits.
4. **System-Level Profiling:** We'll monitor and log the system behaviour under these varying configurations, measuring metrics such as end-to-end iteration time, network I/O overhead, and node-level resource utilisation.

Background

To understand this project, we need to look at three main things. First, Kubernetes manages our containers (called Pods) and runs them on different physical or virtual machines (called Nodes)[1]. Second is PyTorch Distributed Data Parallel (DDP)[3]. When we train a model across multiple Pods, DDP ensures that all Pods share their learning updates (also called gradients) with the rest of the network. Third is the ELECTRA model itself [2]. Unlike typical models, ELECTRA trains two networks simultaneously: a Generator and a Discriminator. This means it has to send twice as much data across the network, making it incredibly sensitive to any network lag.

Methodology

Our system uses a simple Master-Worker setup. The Master pod coordinates training, and the Worker pod handles the heavy lifting. They find and talk to each other using a Kubernetes Service. To test how the network affects performance, we change the scheduling rules. In our first setup, we use a rule called `podAffinity` to force Kubernetes to place both the Master and the Worker on the same physical machine. This simulates a very fast, local network. In our second setup, we use `podAntiAffinity` to force them onto completely different machines. This simulates a slower network where data actually has to travel across cables. The following flowchart outlines the methodological approach to our work.

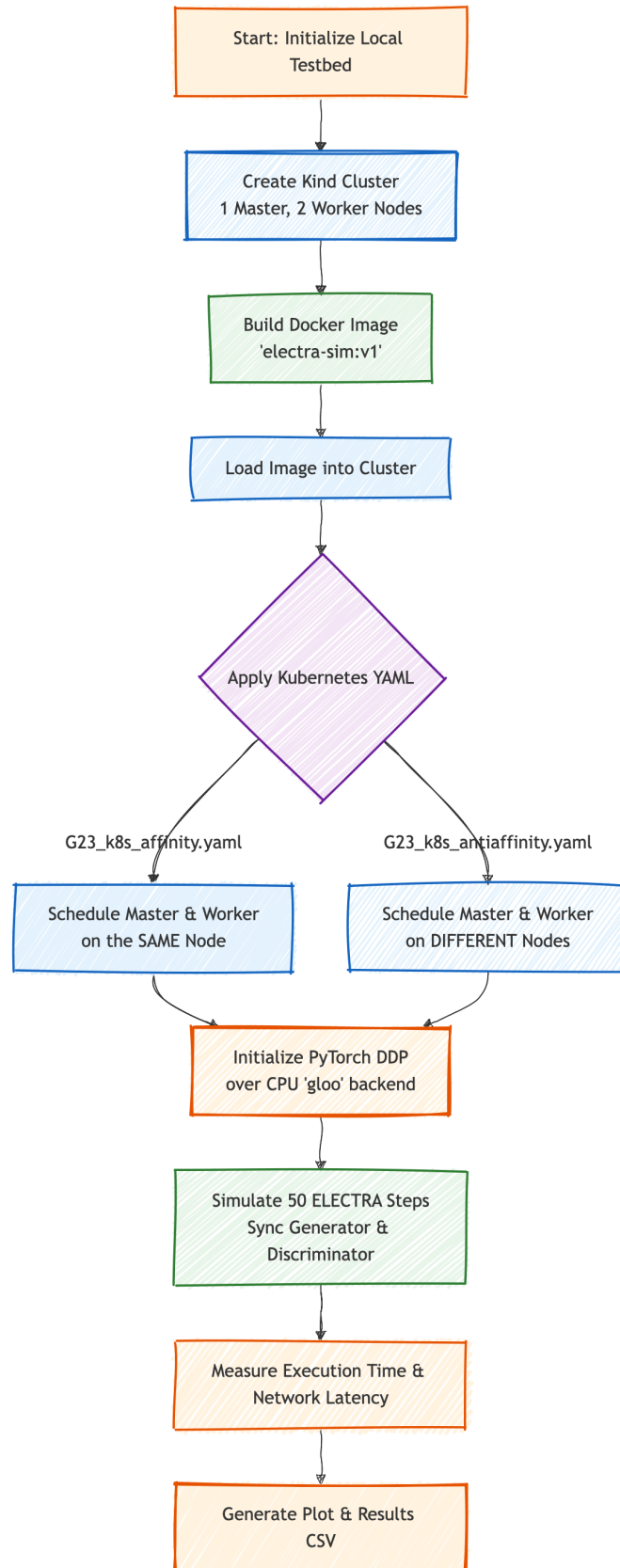


Figure 1: System architecture and execution workflow for the distributed ELECTRA training simulation under varying scheduling policies.

(Initial) Implementation

For this first phase, we built a working local simulation. We used a tool called "kind" (Kubernetes IN Docker) to create a fake multi-node cluster with one primary node and two worker nodes. Then, we wrote a PyTorch script that runs 50 training steps on the CPU using the ELECTRA model. We packed this script into a Docker container using a lightweight Python image so it could run anywhere. Finally, we wrote the Kubernetes YAML files to deploy these containers and explicitly control whether they sit together or apart.

Evaluation & Observations

We ran our experiments on an Apple M1 Mac mini using the local "kind" cluster as our testbed. Our main question was: "How much does putting pods on different nodes slow down the training?" To find out, we measured the total time required to complete the 50 training steps.

When we forced the pods onto the same node (Affinity), the training finished in 11.98 seconds. When we forced them onto different nodes (Anti-Affinity), it took 14.74 seconds.

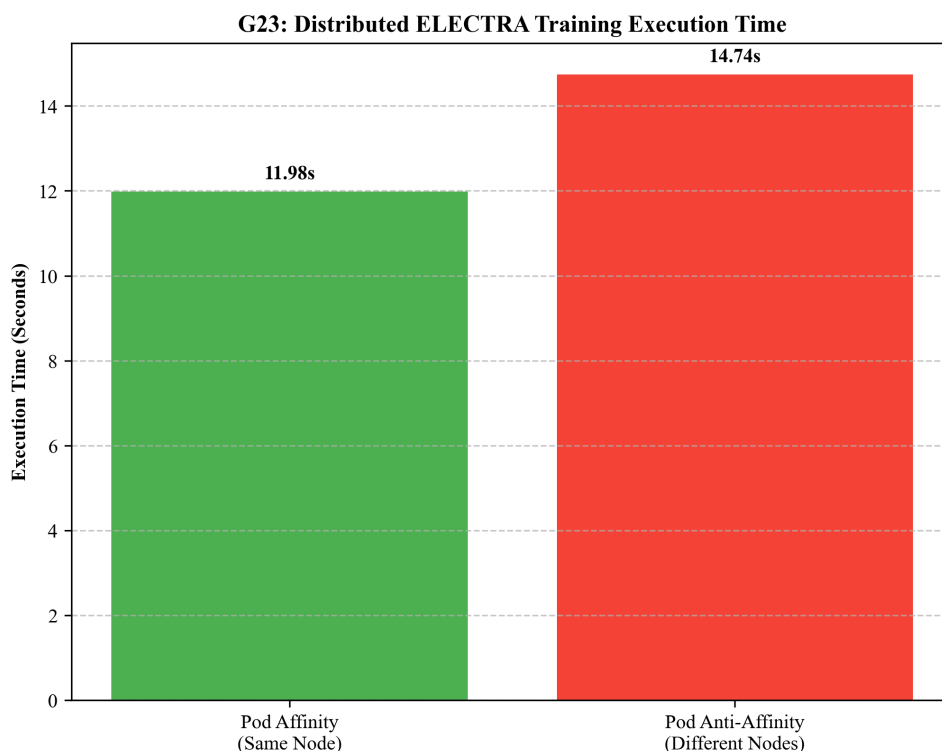


Figure 2: Performance comparison of Distributed ELECTRA training under varying pod placement strategies.

This shows a clear 23% slowdown. This proves our initial thought: simply changing the Kubernetes scheduling policy creates a significant bottleneck because the data takes much longer to travel between machines.

Outcomes & Conclusion

In Phase 1, we successfully built and tested a containerised distributed training environment. Our initial results clearly show that the default, network-blind scheduling in Kubernetes causes

a real performance drop for heavy workloads like ELECTRA. Just moving a pod to a different node slowed our simulation by 23%. For the final project submission, we plan to explore this further, possibly by adding more worker nodes or testing what happens when we restrict the CPU and network resource limits.

Code Availability:

The complete source code, Kubernetes manifest files, and simulation scripts for this project phase are publicly available on GitHub at:

https://github.com/debarshiroy-056/G23_partA_k8s_llm

References

- [1] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.
- [2] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. Electra: Pre-training text encoders as discriminators rather than generators. In *International Conference on Learning Representations*, 2020.
- [3] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, et al. Pytorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12):3005–3018, 2020.